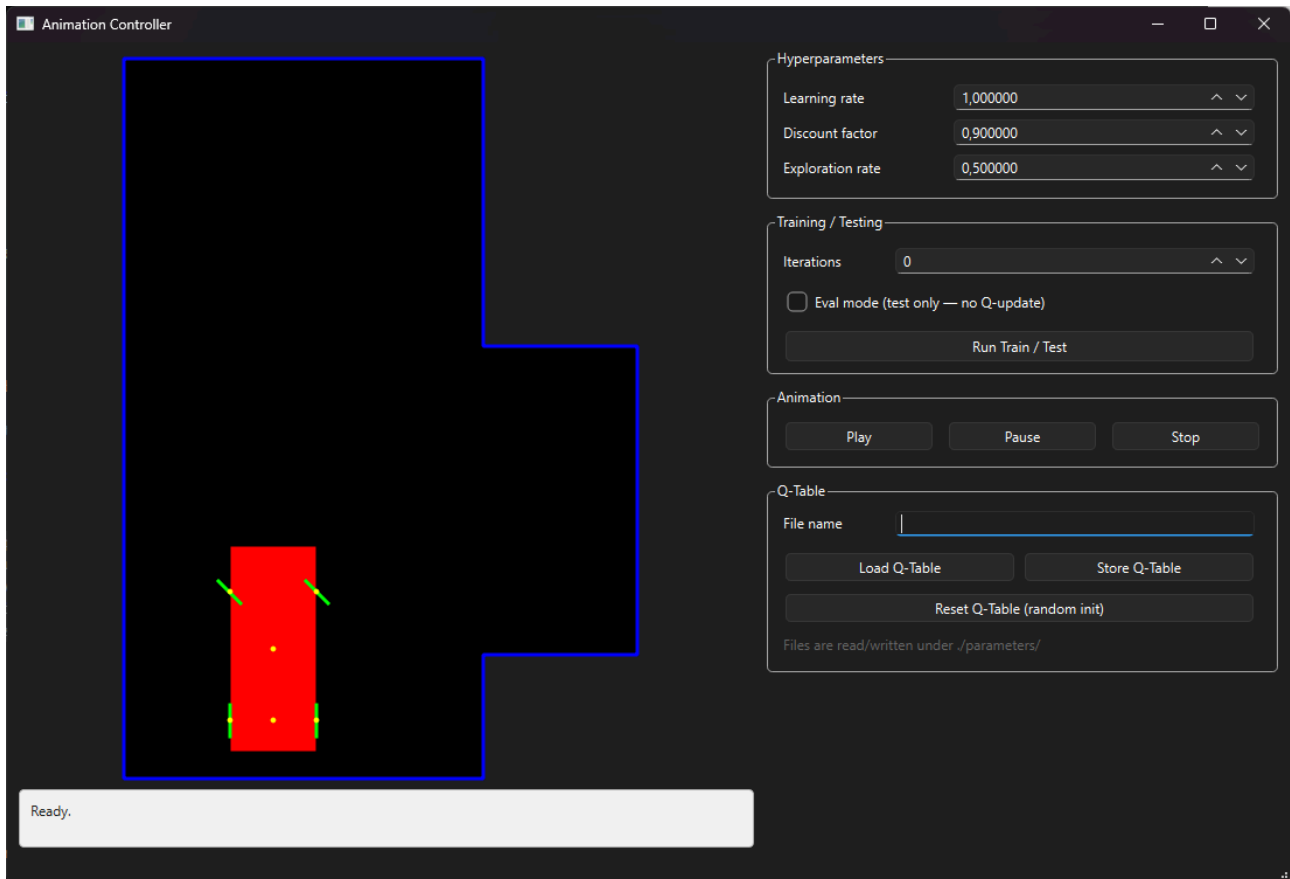


Car Parking with Q-learning

Daniele Pisani

daniele.pisani@santannapisa.it

2026-05-11



Abstract

Autonomous parking is a complex task that requires an agent to learn how to control a car in a constrained environment. In this project, we implemented a simple 2D simulation of a car and trained a Q-learning agent to park it in a designated spot. The car's state is defined by its position and orientation while the action space consists of discrete steering angles and acceleration commands. We experimented with different reward functions and hyperparameters to optimize the learning process. The results show that the agent successfully learns to park the car in the target spot after sufficient training episodes, demonstrating the effectiveness of Q-learning in this context.

Index

1 Introduction	3
2 Physical Modeling	3
2.1 State and inputs	3
2.2 Exact bicycle kinematics	3
2.3 Linearized model	4
2.4 Body geometry and collisions	4
3 Learning architecture	5
3.1 State and action space	5
3.2 Reward and termination	6
3.3 Q-learning update	6
4 Implementation	6
4.1 Tools	6
4.2 Organization	7
4.3 Hyperparameter	8
5 Experiments and results	8
5.1 Optimal Hyperparameters configuration found fot the project . . .	8
6 Conclusions	10
7 Further Work	10

1 Introduction

The aim of this project is training a small Q-learning model to park a car in a designated spot. Here, the car is a simplified 2D model, designed as a rectangle with a front pair of rotating wheels, and a back pair of fixed wheels. The states of the car are defined by its position and orientation. The action space consists of discrete steering angles and speed commands. The environment is a simple parking lot with a designated parking spot. The agent receives rewards based on its ability to successfully park within it while avoiding collisions in as little time as possible.

The choice of method is deliberately conservative. Tabular Q-learning is the simplest possible reinforcement-learning algorithm with provable convergence on finite Markov decision processes, and it serves as a transparent baseline against which any more sophisticated approach can be compared. The cost of this simplicity is paid in the discretization step: the continuous pose of the car must be projected onto a finite grid, and every design decision in Section 2 and Section 3 is shaped by the trade-off between resolution and table size. The remainder of this paper is organized as follows. Section 2 describes the kinematic-bicycle simulator and the geometry of the parking lot. Section 3 defines the discrete state and action spaces, the reward function, and the Q-learning update used by the agent. Section 4 discusses how the simulator and the learner are integrated into a single Qt application, which hyperparameters are exposed, and how the Q-table is persisted. The final two sections summarize qualitative observations and possible extensions.

2 Physical Modeling

The car's dynamics are modeled using a simplified kinematic bicycle model, which captures the essential behavior of a car while being computationally efficient. The speed is modeled instantaneously, meaning that the car can accelerate or decelerate to any speed within a single time step. The steering is also modeled as instantaneous, allowing for discrete changes in the steering angle. The fact that the wheels, specifically the front ones, are not on the edge of the car, but rather a few centimeters inwards, must be taken into account in the model to allow for the parking to be possible.

2.1 State and inputs

The car's pose is described by the triple (x, y, θ) , where (x, y) is the position of the geometric centre of the rectangular body and θ is the heading angle in world coordinates. At each step the controller issues two signals: a longitudinal speed v and a front-wheel steering angle δ , both drawn from a discrete set symmetric around 0.

Let L_c be the total length of the car, w_c its width, and o_f, o_r the front and rear overhangs. The effective wheelbase, i.e. the distance between the front and rear axles, is then

$$L = L_c - o_f - o_r.$$

For the exact kinematic model, the rear-axle contact point (x_r, y_r) is located at a distance $\frac{L_c}{2} - o_r$ behind the body centre along the heading direction:

$$\begin{pmatrix} x_r \\ y_r \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \left(\frac{L_c}{2} - o_r \right) \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}.$$

2.2 Exact bicycle kinematics

With a non-zero steering angle, the car traces a circular arc around the instantaneous centre of rotation (ICR), which lies on the line perpendicular to the heading at the rear axle, at a signed distance $\frac{L}{\tan \delta}$. Over a time step Δt , the swept arc angle is

$$\alpha = \frac{\Delta t v \tan \delta}{L},$$

and the ICR is

$$\begin{pmatrix} x_{cr} \\ y_{cr} \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \left(\frac{L_c}{2} - o_r \right) \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} - \frac{L}{\tan \delta} \begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix}.$$

The new pose is obtained by rigidly rotating (x, y, θ) around (x_{cr}, y_{cr}) by α :

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} x - x_{cr} \\ y - y_{cr} \end{pmatrix} + \begin{pmatrix} x_{cr} \\ y_{cr} \end{pmatrix}, \quad \theta' = \theta + \alpha.$$

When $\delta = 0$ the ICR escapes to infinity, so the update reduces to pure translation along the heading:

$$x' = x + \Delta t v \cos \theta, \quad y' = y + \Delta t v \sin \theta, \quad \theta' = \theta.$$

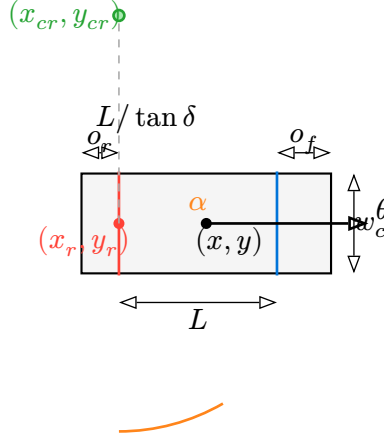


Figure 1: Kinematic bicycle model. The body of length L_c and width w_c has wheelbase $L = L_c - o_f - o_r$, with overhangs o_f and o_r at the front and rear. The rear axle (x_r, y_r) lies at distance $\frac{L_c}{2} - o_r$ behind the geometric centre (x, y) . For non-zero steering δ , the body rotates around the instantaneous centre of rotation (x_{cr}, y_{cr}) , perpendicular to the heading at signed distance $L / \tan \delta$ from the rear axle; the swept arc over one timestep is α .

2.3 Linearized model

A simpler integrator is also implemented (selected by the APPROX_MOTION flag in the configuration). It tracks a rear reference point $(\tilde{x}_r, \tilde{y}_r)$ taken at the midpoint of the rear bumper rather than at the rear axle (the overhang o_r is ignored here, so this point is offset by $\frac{L_c}{2}$ instead of $\frac{L_c}{2} - o_r$):

$$\begin{pmatrix} \tilde{x}_r \\ \tilde{y}_r \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \frac{L_c}{2} \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}.$$

This point is translated straight along the current heading and the heading is updated independently, dropping the curvature coupling:

$$\begin{pmatrix} \tilde{x}_r' \\ \tilde{y}_r' \end{pmatrix} = \begin{pmatrix} \tilde{x}_r \\ \tilde{y}_r \end{pmatrix} + \Delta t v \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}, \quad \theta' = \theta - \frac{\Delta t v \sin \delta}{L_c}.$$

This trades geometric fidelity for speed: it replaces $\tan \delta$ with $\sin \delta$ and uses the full body length L_c in place of the wheelbase L , so the predicted curvature is biased for large steering angles. The exact model is the default.

2.4 Body geometry and collisions

The four corners (x_i, y_i) , $i = 1, \dots, 4$, of the car rectangle are reconstructed independently from the pose, each as the body centre plus a half-length offset along the heading and a half-width offset perpendicular to it:

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} \pm \frac{L_c}{2} \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} \pm \frac{w_c}{2} \begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix},$$

with the four sign combinations giving the four vertices. A configuration is admissible when all four corners lie inside the parking-lot polygon and the body does not swallow either of the two re-entrant corners of the slot pocket. The car is considered parked when all four corners lie inside the slot quadrilateral and the heading satisfies

$$|\theta - (-\frac{\pi}{2})| \leq \frac{\pi}{6}.$$

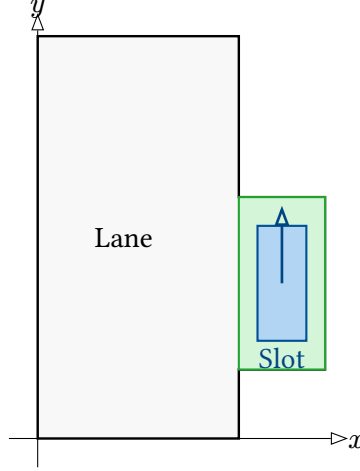


Figure 2: Parking lot polygon (light grey) and target slot quadrilateral (green), drawn from `configuration/default.conf` with $W_{\text{env}} = 10$, $L_{\text{env}} = 14$, $W_{\text{slot}} = 3$, $L_{\text{slot}} = 6$, and $\text{FREE_PARK} = 0.4$. The blue rectangle shows the canonical parked configuration, with heading $\theta = -\frac{\pi}{2}$ in the simulator's y -down convention (mirrored here for readability so that y points up).

3 Learning architecture

This chapter describes how the continuous parking task introduced in Section 2 is reformulated as a discrete decision process suitable for tabular Q-learning. The three aspects are the discretization of the state space obtained by discretizing the pose of the car, a discrete action space obtained by setting discrete values for speed and steering, and a sparse reward function that has only three possible outcomes per step.

3.1 State and action space

The state of the agent is the discretized pose of the car. The continuous triple (x, y, θ) is binned independently into X_{DIVIDE} position bins along the lot width, Y_{DIVIDE} position bins along its length, and Θ_{DIVIDE} angular bins covering $[0, 2\pi)$, yielding

$$N_{\text{states}} = X_{\text{DIVIDE}} \cdot Y_{\text{DIVIDE}} \cdot \Theta_{\text{DIVIDE}}$$

discrete states. The state index packs the three components with θ as the fastest-varying axis, so that nearby orientations occupy contiguous indices in the table:

$$s = i_{\theta} + \Theta_{\text{DIVIDE}}(i_y + Y_{\text{DIVIDE}} i_x).$$

The bin indices i_x, i_y, i_{θ} are obtained by integer division of each coordinate by the corresponding section width, with θ wrapped into $[0, 2\pi)$ before binning. Cells whose index would overflow are clamped to the nearest valid bin so that no transition can ever produce an out-of-range state.

The action space instead is the Cartesian product of a discrete speed set and a discrete steering set, both built by a small helper that produces n values symmetric around zero with a fixed spacing. Speeds are spaced in metres per second, steering values are spaced by a configurable angular unit and converted from degrees to radians before use. The total number of actions is

$$N_{\text{actions}} = N_{\text{speed}} \cdot N_{\text{steering}},$$

The presence of a zero entry in both sets is essential: it lets the agent stand still or drive straight, which is required for the fine-grained alignment manoeuvres performed near the slot.

3.2 Reward and termination

The reward function is deliberately sparse and depends only on the geometric classification of the resulting pose. After applying a transition, the environment classifies s' into one of three regimes and emits a constant reward:

$$r(s, a, s') = \begin{cases} R_{\text{hit}} & \text{if there's a collision with the lot boundaries} \\ R_{\text{park}} & \text{if the car is parked} \\ R_{\text{nothing}} & \text{otherwise.} \end{cases}$$

With $R_{\text{hit}} < 0$, $R_{\text{park}} > 0$ and R_{nothing} either zero or a small negative time penalty, this scheme penalizes collisions, rewards success, and lets the discount factor γ implicitly favour shorter trajectories. The “parked” predicate combines two conditions, a geometric one and an angular one: the four corners of the body must lie inside the slot quadrilateral and the heading must satisfy $|\theta - (-\frac{\pi}{2})| \leq \frac{\pi}{6}$, as derived in Section 2.

3.3 Q-learning update

The agent maintains a tabular action-value function $Q \in \mathbb{R}^{N_{\text{states}} \times N_{\text{actions}}}$, initialized to zero. At every model step the standard tabular Q-learning update is applied:

$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right),$$

with learning rate η and discount factor γ . Action selection uses an ε -greedy policy:

$$\pi(s) = \begin{cases} \text{uniform random action with probability } \varepsilon \\ \arg \max_a Q(s, a) & \text{with probability } 1 - \varepsilon. \end{cases}$$

During evaluation the random branch is suppressed by setting $\varepsilon = 0$. This is exposed in the user interface through an eval checkbox: ticking it bypasses the Q-update inside the iteration step and forces purely greedy action selection, turning the agent into an exploiter of the current table without overwriting it.

Both η and ε decay over training. Each parameter is configured by a maximum value, a minimum value, and a half-life H expressed in model iterations. Every iteration the parameter is multiplied by a constant ratio $\rho = 2^{-\frac{1}{H}}$ derived from the half-life, so that after H iterations the gap between the current value and the asymptotic floor has halved:

$$\eta_t - \eta_{\min} \approx (\eta_{\max} - \eta_{\min}) \rho_{\eta}^t, \quad \rho_{\eta} = 2^{-\frac{1}{H_{\eta}}},$$

and analogously for ε . This schedule lets the agent explore aggressively early in training while gradually committing to the policy implied by the current Q-values, and prevents stale large-step updates from overwriting a good table once the policy has approximately converged.

4 Implementation

The full project is implemented as a single Qt 5/6 desktop application written in C++. The same binary handles training and visualization, and the user interface allows for switching between the two modes. There is no external machine-learning framework: the Q-table, the simulator, and the rendering pipeline are all hand-written in a few thousand lines of code.

4.1 Tools

Building requires Qt (modules core, gui and widgets) together with qmake and a make-compatible toolchain. From the project root the standard sequence is `qmake carparking.pro` to generate the Makefile, then `make` (or `mingw32-make` on Windows, `nmake` under MSVC) to compile, and finally `./bin/carparking` to run. Object

files are emitted to `build/` and the executable to `bin/`, controlled by `OBJECTS_DIR` and `DESTDIR` in the project file.

The application also relies on three paths resolved relative to the working directory: `configuration/default.conf` is read once at startup, `parameters/` stores serialized Q-tables, and `log/log12.txt` receives the periodic training log written by the headless training loop.

4.2 Organization

The codebase is organized around three components that mirror the natural separation between simulator, learner, and presentation:

- `Environment` and `CarState` (`src/environment.{h,cpp}`) implement the kinematic model described in Section 2, the parking-lot polygon, the collision and parking checks, the reward function, and the state discretization.
- `QLearningModel` (`src/q_learning.{h,cpp}`) owns the flat Q-table as a `vector<vector<float>>` of shape $[N_{\text{states}}, N_{\text{actions}}]$, the ϵ -greedy action selection, the tabular update, and the schedules for η and ϵ .
- `MainWindow` (`src/mainwindow.{h,cpp}` and `mainwindow.ui`) implements the UI of the application, the rendering of the parking lot and the car, the buttons to trigger training and animation, the load and store buttons for the Q-table, and the `eval` checkbox that toggles between training and pure greedy evaluation. Also creates the instances of `Environment` and `QLearningModel` and orchestrates the interaction between them.

The application supports two distinct execution paths, each driven by its own button. *Animated mode* advances the simulation with a `QTimer`: every tick steps the environment by one timestep, and the model is updated only every `time_ratio` ticks, which lets the visual pace differ from the learning pace. This path is meant for visualizing the agent's behavior and is accessible through the animation section in the UI. *Headless training*, by contrast, once started, enters a tight for loop over model iterations with no timer, alternating one Q-learning update with one environment step on every iteration. This is the only path that prints aggregated statistics to the standard output and writes the structured log to `log/log12.txt` every `LOG_FREQ` iterations, and is the path used to actually perform the proper training and testing.

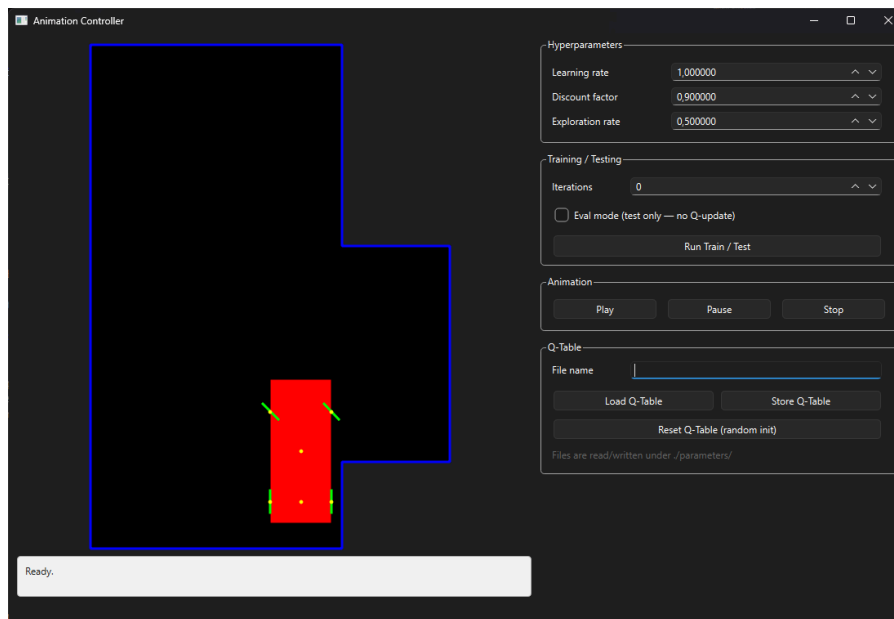


Figure 3: The Qt application at runtime. The left side renders the parking lot polygon and the current pose of the car; the right side groups the controls for both execution paths and the load and store buttons for the Q-table.

4.3 Hyperparameter

All numerical parameters are externalized to `configuration/default.conf`, parsed once at construction time into a `map<string, string>`. Because the file is read only once, every change requires restarting the application.

The parameters fall into five groups:

- *Car and lot geometry*: `LEN_CAR`, `WIDTH_CAR`, `FRONT_OVERHANG`, `REAR_OVERHANG`, `LEN_ENV`, `WIDTH_ENV`, `LEN_SLOT`, `WIDTH_SLOT`, `FREE_PARK`. These determine the rectangle in Section 2 and the polygon of the parking lot.
- *Reward shaping*: `REWARD_FOR_HIT`, `REWARD_FOR_PARK`, `REWARD_FOR_NOTHING`. The three constants entering the reward function above.
- *Learning hyperparameters*: `Q_LR_MAX`, `Q_LR_MIN`, `Q_LR_HL` for the learning-rate schedule, `ER_MAX`, `ER_MIN`, `ER_HL` for the exploration-rate schedule, and `DISCOUNT_FACTOR` for γ .
- *Discretization*: `X_DIVIDE`, `Y_DIVIDE`, `THETA_DIVIDE` for the state-space grid, `N_SPEED_ACTIONS`, `N_STEERING_ACTIONS`, `STEERING_UNITY` for the action grid.
- *Execution*: `APPROX_MOTION` selects the integrator of Section 2 MSEC, `ANIMATION_SPEED`, and `TIME_RATIO` control the timing of the animated mode.

The Q-table itself is saved and stored as plain text files, with one row per state and `action_count` floats per row separated by spaces.

5 Experiments and results

At the start of each run, the car position is always reset to a random initial position, and the heading is always reset to $-\frac{\pi}{2}$, keeping the initial-state distribution more narrow and predictable across runs. In this way the model is capable of achieving a very high accuracy score even with the tight discretizations required.

5.1 Optimal Hyperparameters configuration found for the project

After exploring the parameter ranges described in Section 4, the configuration shipped in `configuration/default.conf` emerged as a reliable working point for the parking task. With these values the agent reaches a per-window success ratio above 98% after roughly ten million model iterations, as documented in the baseline curve at the end of this section. Table 1 reports the values used; they serve as the reference against which the variants presented below are compared.

Parameter	Value	Role
<i>Reward shaping</i>		
REWARD_FOR_HIT	−50	collision penalty R_{hit}
REWARD_FOR_PARK	100	success reward R_{park}
REWARD_FOR_NOTHING	−1	per-step time penalty R_{nothing}
<i>Learning algorithm</i>		
DISCOUNT_FACTOR	0.9	discount factor γ
Q_LR_MAX	1.0	initial learning rate η_{max}
Q_LR_MIN	0.001	asymptotic floor η_{min}
Q_LR_HL	$5 \cdot 10^6$	half-life H_η (iterations)
ER_MAX	0.5	initial exploration rate ε_{max}
ER_MIN	0.0001	asymptotic floor ε_{min}
ER_HL	$2 \cdot 10^6$	half-life H_ε (iterations)
<i>State and action discretization</i>		
X_DIVIDE	40	position bins along x
Y_DIVIDE	56	position bins along y
THETA_DIVIDE	48	heading bins on $[0, 2\pi)$
N_SPEED_ACTIONS	5	discrete speed levels
N_STEERING_ACTIONS	7	discrete steering angles
SPEED_UNITY	1	speed spacing (m/s)
STEERING_UNITY	15	steering spacing (degrees)
<i>Simulator</i>		
APPROX_MOTION	0	exact bicycle kinematics

Table 1: Default hyperparameter configuration, treated as the optimal working point in the experiments that follow. Geometry, animation, and window parameters are omitted because they do not influence the learned policy.

The configuration in Table 1 is not the only viable working point: several of its entries can be varied without breaking the agent, and different values produce qualitatively different learning curves. The two parameters with the largest influence on convergence are the initial learning rate η_{max} (Q_LR_MAX) and the initial exploration rate ε_{max} (ER_MAX), both subject to the half-life decay schedule introduced in Section 3. To make this dependence visible, the curve under the optimal configuration is shown first, and is then compared with two variants in which a single parameter has been halved while all other entries of Table 1 are kept fixed.

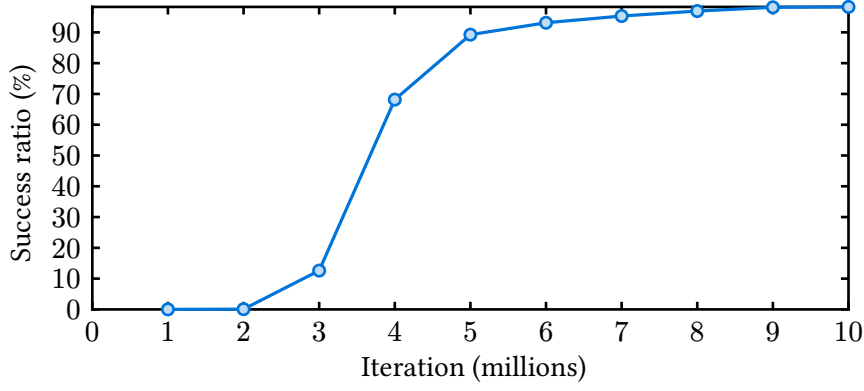


Figure 4: Per-window success ratio from a representative 10-million-iteration training run with the default configuration. Each point reports, over the most recent million iterations, the fraction of terminal events (hit + success) that ended in a successful park. The curve rises as the schedule on ε pushes the agent from exploration into exploitation of the learned table.

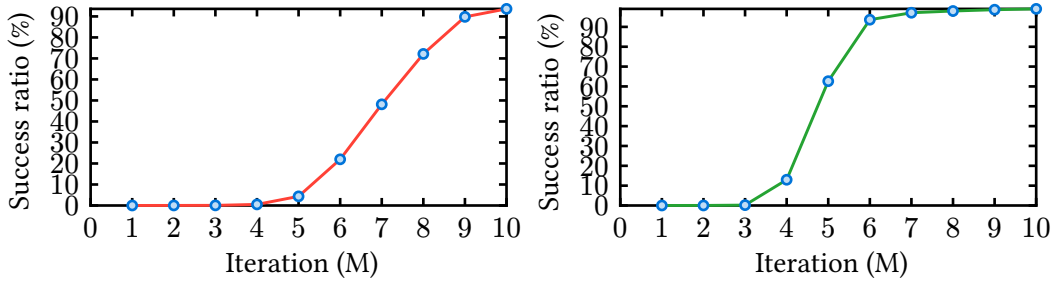


Figure 5: Same experiment as Figure 4 with a single parameter halved, all other parameters as in Table 1. **Left:** Q_LR_MAX reduced from 1.0 to 0.5, final success ratio 93.6%. **Right:** ER_MAX reduced from 0.5 to 0.25, final success ratio 99.1%.

The two perturbations confirm that η and ε play opposite roles: halving $\eta_{\max}$ slows reward propagation and leaves the agent short of convergence within the 10-million budget, while halving $\varepsilon_{\max}$ lets the agent exploit the already-informative reward signal sooner and marginally improves the steady-state success rate. The default $\eta_{\max} = 1.0$ is therefore close to the largest useful initial step, whereas $\varepsilon_{\max} = 0.5$ is slightly conservative on this task but remains the safer default for coarser discretizations or less benign reward landscapes.

6 Conclusions

This project shows that a standard tabular Q-learning agent, with a properly discretized state space, can learn a parking policy for a kinematic-bicycle car in a constrained 2D lot, with learner, simulator, and visualization fitting in a single Qt application of a few thousand lines and no external machine-learning dependencies. The exercise also exposes the ergonomic ceiling of tabular methods: the three discretization axes contribute multiplicatively to the state count, and the fixed reset orientation of $-\frac{\pi}{2}$ further narrows the training distribution. The controller is nevertheless faithful to the textbook formulation of Q-learning and provides a transparent baseline against which any subsequent extension can be compared on equal terms.

7 Further Work

The most obvious extension is replacing the tabular Q-table with a function approximator: a small DQN-style network would eliminate the multiplicative blow-up in the state count, allow continuous (x, y, θ) inputs, and generalize between nearby poses. Three complementary changes would round out the picture: bounded acceleration and steering rate instead of instantaneous commands; random reset orientations (the commented-out line in `set_random_carstate` already suggests this); and a structured per-window log paired with a “benchmark” mode reporting a numerical success rate over N random-start episodes.